ChatGPT

Acquisto future EUR USD

Tu

mi esponi entrambe le situazioni poi vedo quella piu'conveniente da modellare

ChatGPT

Certamente. Ti descrivo entrambi i modelli partendo dallo stesso esempio: un cliente della banca vuole acquistare **10 future EUR/USD (CME 6E)**.

La differenza non è soltanto operativa: cambia il rapporto contrattuale con il cliente, la gestione delle posizioni, il rischio assunto dalla banca e il modello contabile.

1. Modello broker: la banca intermedia l'operazione sul CME

Agency / Execution

In questo modello la banca agisce come intermediario. Il cliente vuole acquistare un future e la banca trasmette l'ordine al mercato.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

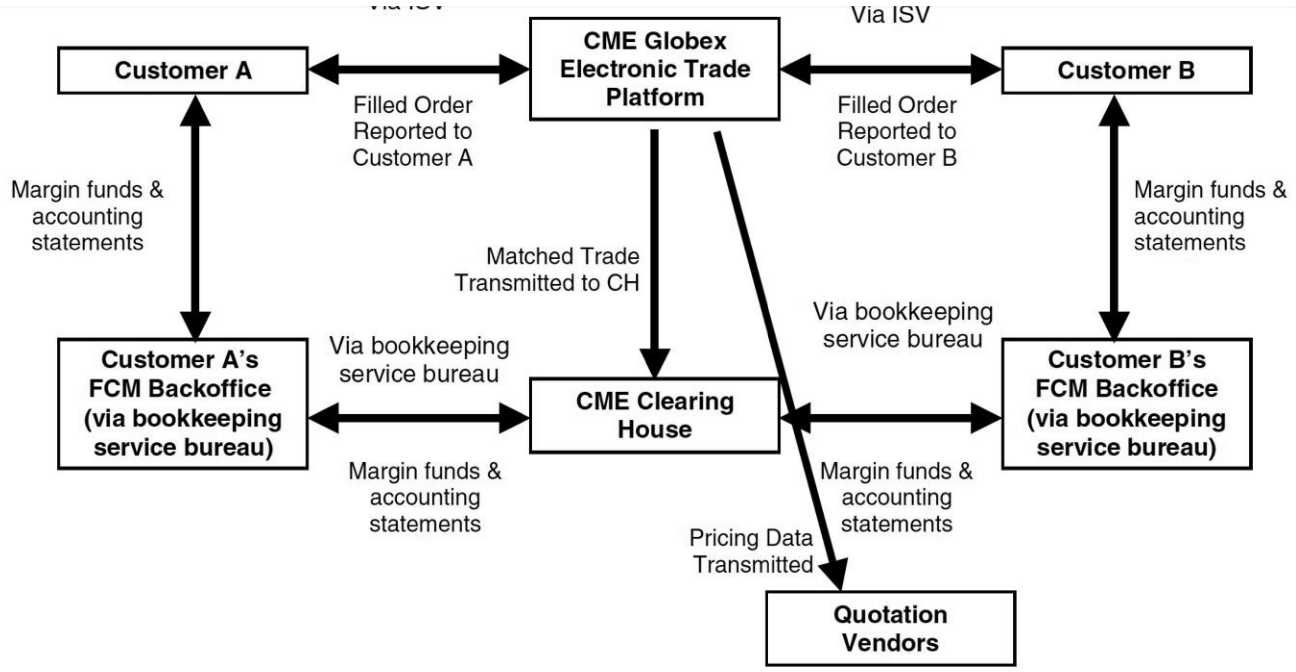


EXHIBIT 1.16 Anatomy of an Electronic Futures Transaction

Flusso operativo

1. Il cliente richiede l'acquisto di 10 contratti 6E.
2. La banca verifica disponibilità, abilitazioni e margini richiesti.
3. La banca trasmette l'ordine al proprio broker/FCM.
4. Il broker esegue l'ordine sul CME.
5. La posizione viene attribuita al conto di clearing del cliente, direttamente o attraverso una struttura omnibus secondo il modello di custodia e clearing.
6. La banca aggiorna la posizione del cliente nel proprio sistema.

Supponiamo che il future venga acquistato a 1,08500.

Elemento	Valore
Contratti	10
Nominale per contratto	125.000 EUR
Nominale complessivo	1.250.000 EUR
Prezzo di acquisto	1,08500

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Commissione banca	20 EUR

La banca non assume il rischio di mercato del future per conto proprio: il rischio economico appartiene al cliente.

Come guadagna la banca?

Principalmente attraverso:

- Commissioni di esecuzione.
- Commissioni di clearing e custodia, eventualmente maggiorate.
- Commissioni per il servizio di accesso al mercato.

Non necessariamente applica un ricarico sul prezzo del future.

Gestione delle posizioni

Avresti due livelli distinti:

- Posizione del cliente: 10 contratti long.
- Posizione presso il clearing broker: posizione riferibile al cliente o al relativo conto di clearing.

Non devi necessariamente avere una posizione proprietaria della banca.

Contabilizzazione

La banca non contabilizza il future come posizione di trading proprietaria.

Contabilizza invece:

- Commissioni attive.
- Eventuali commissioni passive verso il broker.
- Margini e collateral ricevuti dal cliente come attività/passività secondo la loro natura.
- Eventuali crediti/debiti derivanti dai flussi di regolamento.

Il variation margin del future è economicamente a carico o a favore del cliente e deve essere gestito coerentemente con la struttura contrattuale e di clearing.



Porta la tua curiosità più lontano

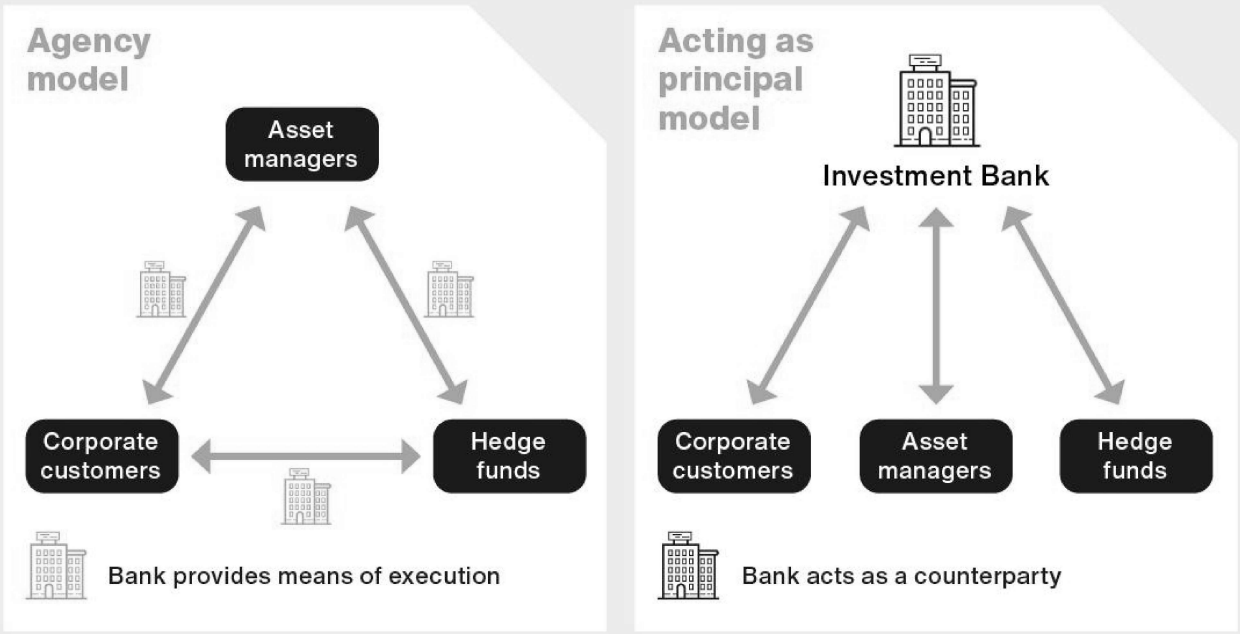
Scansiona per scaricare l'app ChatGPT



2. Modello dealer: la banca è la controparte del cliente

Principal / Market Making

Questo è un modello completamente diverso. La banca non si limita a trasmettere un ordine: vende direttamente al cliente un'esposizione finanziaria equivalente a quella del future.



Flusso operativo

Il cliente richiede 10 future EUR/USD.

La banca consulta il mercato CME e costruisce una propria quotazione.

Esempio:

Elemento	Valore
Prezzo CME ask	1,08500
Prezzo offerto dalla banca	1,08520
Contratti	10

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Spread	0,00020

Il cliente acquista dalla banca a 1,08520.

La banca assume una posizione short nei confronti del cliente.

A questo punto può scegliere come gestire il rischio.

Caso A: copertura immediata sul CME

La banca acquista 10 contratti CME a 1,08500.

Cliente
Long 10 contratti sintetici a 1,08520

↓↑

Banca
Short 10 contratti verso il cliente
Long 10 contratti CME

↓↑

CME
Long 10 contratti della banca

La banca ha neutralizzato il rischio di mercato, salvo rischi operativi, di basis, di liquidità e di esecuzione.

Il guadagno iniziale teorico è:

$$125.000 \times 10 \times 0,00020 = 250\ EUR$$

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Questi 250 EUR rappresentano lo spread lordo, prima dei costi di copertura e degli altri costi operativi.

Caso B: compensazione tra clienti

Supponiamo che un secondo cliente voglia vendere 6 contratti.

La banca ha:

Cliente	Posizione banca
Cliente A acquista 10	Short 10
Cliente B vende 6	Long 6
Esposizione netta banca	Short 4

La banca potrebbe acquistare soltanto 4 contratti sul CME per coprirsi.

Questo è un vantaggio importante del modello dealer: la banca può compensare le esposizioni interne prima di andare sul mercato.

Caso C: la banca mantiene il rischio

La banca potrebbe anche decidere di non coprirsi immediatamente, o di coprirsi solo parzialmente.

In questo caso assume una posizione proprietaria e beneficia o subisce le variazioni di mercato.

È una scelta di gestione del rischio, non una necessità operativa.

3. Differenze contabili tra i due modelli

Questa è probabilmente la parte più interessante per la progettazione del tuo applicativo.

Aspetto	Broker	Dealer
Controparte del cliente	Mercato, tramite intermediario	Banca

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Proprietà economica della posizione	Cliente	Cliente ha un derivato verso banca
Posizione proprietaria banca	Non necessaria	Sì, per eventuale hedge
Ricavo principale	Commissioni	Spread e margini commerciali
Rischio di mercato banca	Limitato al servizio	Dipende dalla copertura
Clearing	Per conto cliente	Per conto banca
Variation margin	Gestito per il cliente	Gestito sulla posizione CME della banca
MTM	Sul conto cliente	Sul derivato cliente e sull'hedge

Nel modello dealer devi distinguere due rapporti:

1. Il derivato stipulato tra banca e cliente.
2. Il future CME acquistato dalla banca per coprire il derivato.

Non sono lo stesso strumento, anche se hanno un'esposizione economica simile.

3. Come cambia il modello applicativo

Ora traduciamo i due scenari nella tua architettura Java/Spring Boot, considerando le entità che hai già sviluppato: `FinancialTxn`, `MasterData`, `PositionMasterData`, `PositionDetail`, `AccountingEvent` e il motore contabile.

A. Architettura broker

Qui il cliente possiede una posizione effettiva sul mercato regolamentato.

La transazione deve rappresentare l'operazione eseguita per conto del cliente.

FinancialTxn

Campo	Valore
transactionId	1001
portfolio	Portfolio cliente

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

instrument	CME 6E DEC-26
assetClass	FUTURE
side	BUY
quantity	10
price	1,08500
counterparty	Clearing broker
status	EXECUTED

Aggiungerei alcuni campi per identificare l'esecuzione:

- `executionVenue` : CME
- `brokerId` : intermediario utilizzato
- `clearingAccountId` : conto di clearing
- `exchangeExecutionId` : identificativo dell'esecuzione.

La posizione viene aggiornata nel portfolio del cliente.

PositionDetail

```
Portfolio Cliente
    |
    +-- PositionMasterData
            |
            +-- PositionDetail
                    |
                    +-- Instrument: CME 6E DEC-26
                    +-- QuantityBuy: 10
                    +-- QuantitySell: 0
```

La banca non deve necessariamente creare una `PositionDetail` proprietaria.

Per la contabilità, gli eventi principali saranno:

- TRADE_EXECUTED: registrazione dell'operazione e delle commissioni.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

- MARGIN: gestione del margine iniziale.
- SETTLEMENT: regolazione giornaliera del variation margin.
- TRADE_CANCELED: eventuale chiusura o rettifica.

Attenzione: il margine iniziale non è un costo del future e il variation margin non è un normale flusso di cassa di un FX spot. La contabilizzazione dipende anche dalla struttura del conto di clearing.

B. Architettura dealer

Qui la banca ha due portafogli distinti:

1. Portfolio cliente, che rappresenta il contratto stipulato con il cliente.
2. Portfolio proprietario della banca, che rappresenta la copertura sul mercato.

Per esempio:

Portfolio cliente A

Contratto derivato EUR/USD — LONG 10

Portfolio Proprietario Banca

Future CME 6E DEC-26 — LONG 10

La banca è short verso il cliente e long sul mercato per copertura.

Qui farei una distinzione importante.

Se la banca stipula con il cliente un derivato OTC che replica il comportamento del future, il prodotto non è tecnicamente un future CME: è un contratto bilaterale con payoff equivalente o simile.

Se invece il cliente acquista un vero future CME, la banca può agire da principal nell'esecuzione e nel clearing, ma la posizione exchange-traded del cliente resta distinta dalla posizione proprietaria della banca.

Questa distinzione è importante anche per il tuo modello anagrafico.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Le transazioni

Avresti due `FinancialTxn`:

Transazione 1 – Contratto cliente

```
transactionId: 2001
portfolio: Cliente A
instrument: FX_FUTURE_OTC
side: BUY
quantity: 10
price: 1.08520
counterparty: Banca
status: EXECUTED
```

Transazione 2 – Copertura sul mercato

```
transactionId: 2002
portfolio: Proprietary Portfolio
instrument: CME 6E DEC-26
side: BUY
quantity: 10
price: 1.08500
counterparty: Clearing Broker
executionVenue: CME
status: EXECUTED
```

Le due operazioni dovrebbero essere collegate mediante una relazione di copertura, non attraverso un legame rigido 1:1.

Per esempio, potresti introdurre:

`HedgeAllocation`

Campo	Descrizione
hedgeAllocationId	Identificativo

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

clientTxnId	Transazione cliente
hedgeTxnId	Transazione di copertura
allocatedQuantity	Quantità allocata
allocationDate	Data allocazione

Questa struttura consente anche di gestire coperture parziali e aggregate.

Contabilità dealer

Qui il motore contabile deve gestire due posizioni economiche distinte:

- Il derivato OTC verso il cliente.
- Il future CME detenuto dalla banca.

Entrambi devono essere valutati e contabilizzati, ma con regole diverse.

Per esempio, se il prezzo di mercato passa da 1,08500 a 1,08600:

- Il cliente ha un utile sul derivato OTC.
- La banca ha una perdita sul derivato verso il cliente.
- La banca ha un utile sul future CME.

Il risultato economico netto della banca, al netto dei costi e degli eventuali disallineamenti, rimane sostanzialmente neutrale.

4. Quale modello sceglierei per il tuo applicativo?

Personalmente adotterei un'architettura che supporti entrambi, ma partirei dal modello broker.

Il motivo è che il future CME è già uno strumento standardizzato, con caratteristiche contrattuali, scadenze, moltiplicatori, margini e modalità di settlement definiti dall'exchange.

Non introdurrei un nuovo strumento finanziario soltanto per rappresentare il rapporto commerciale tra banca e cliente.

Imposterei invece tre concetti indipendenti:



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



1. Instrument

Rappresenta lo strumento finanziario vero e proprio:

- CME 6E DEC-26
- CME 6E MAR-27
- CME 6E JUN-27

2. FinancialTxn

Rappresenta un'operazione finanziaria. Aggiungerei un attributo che identifica la natura dell'operazione:

```
public enum ExecutionModel {
    AGENCY,       // Esecuzione per conto cliente
    PRINCIPAL     // Operazione in conto proprio
}
```

Non confonderei però `ExecutionModel` con il tipo di contratto: un future exchange-traded e un derivato OTC devono restare strumenti distinti.

3. Hedge relationship

Rappresenta il collegamento tra l'esposizione verso il cliente e la relativa copertura.

La modellerei indipendentemente dalle transazioni, perché una copertura può essere modificata, frazionata o riallocata nel tempo.

5. Una considerazione che ritengo importante

Non confonderei mai questi tre elementi:

Elemento	Significato
Ordine cliente	Intenzione di acquistare o vendere
Esecuzione mercato	Operazione effettivamente conclusa sul CME
Posizione	Esposizione finanziaria risultante

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Sono tre oggetti diversi.

Per esempio, un ordine cliente di 10 contratti potrebbe essere eseguito in tre tranche:

- 4 contratti a 1,08500
- 3 contratti a 1,08505
- 3 contratti a 1,08510

Avresti un unico ordine, tre esecuzioni e una posizione risultante di 10 contratti con prezzo medio ponderato.

Questa separazione ti sarà utile anche per gestire cancellazioni, rettifiche e riconciliazioni con il broker.

La mia raccomandazione finale: costruirei inizialmente il modello broker, mantenendo però la possibilità di gestire un portfolio proprietario della banca e le relazioni di copertura. In questo modo non vincoli l'architettura a un solo modello commerciale e puoi introdurre successivamente il dealer senza stravolgere `FinancialTxn`, `PositionDetail` e il motore contabile.

Tu

quindi nel caso brooker terrei solo la posizione cliente? E la contoprate e'la banca?

ChatGPT

Sì, nel modello **broker puro** manterresti la posizione del cliente, senza creare una posizione proprietaria della banca per il future.

C'è però una distinzione importante sulla controparte.

1. Chi è la controparte?

Nel modello broker, la banca agisce come intermediario e non è la controparte economica dell'operazione.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

La catena è:

```
Cliente
   |
   | Ordine di acquisto
   v
Banca (broker / intermediario)
   |
   | Trasmissione ordine
   v
FCM / Clearing Broker
   |
   v
CME Clearing
```

La controparte centrale del future è CME Clearing, attraverso il meccanismo di novazione e clearing.

Quindi, se vuoi rappresentare correttamente la struttura:

Elemento	Soggetto
Cliente	Titolare della posizione economica
Banca	Broker / intermediario
FCM	Clearing broker
CME Clearing	Controparte centrale (CCP)

La banca non è automaticamente la controparte del cliente.

2. Come valorizzerei FinancialTxn?

Nel tuo applicativo distinguerei almeno due concetti:

- `counterparty` : controparte contrattuale dell'operazione;
- `executingBroker` : intermediario che ha eseguito o trasmesso l'ordine.

Per esempio:

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

```
FinancialTxn
-------------------------
portfolio        = Cliente A
instrument       = CME 6E DEC-26
side             = BUY
quantity         = 10
price            = 1.08500
counterparty     = CME Clearing
executingBroker  = Banca
clearingBroker   = FCM XYZ
executionVenue   = CME
```

Una precisazione: se il conto di clearing è omnibus, la posizione registrata presso il clearing broker può essere intestata alla banca o a un nominee, mentre la banca mantiene la contabilità individuale dei clienti. In questo caso non devi necessariamente indicare CME Clearing come controparte direttamente nei tuoi dati contrattuali: devi rispettare la struttura giuridica e operativa effettiva.

3. Come strutturerei le posizioni

Nel modello broker avresti semplicemente:

```
Portfolio Cliente A
    |
    +-- PositionMasterData
            |
            +-- PositionDetail
                    |
                    +-- CME 6E DEC-26
                    +-- QuantityBuy = 10
                    +-- QuantitySell = 0
```

Nessuna posizione proprietaria della banca per questa operazione.

La banca avrebbe invece una propria contabilità dei servizi di intermediazione e degli eventuali margini/collateral gestiti.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Un suggerimento per il tuo modello dati: eviterei di utilizzare `counterparty` per rappresentare genericamente tutti gli attori della catena operativa. Terrei separati controparte, broker esecutore, clearing broker e mercato di esecuzione. Ti tornerà molto utile anche per futures su tassi, obbligazioni e commodity.

Tu

mi servirebbero ulteriori dettagli sulla scomposizione controparte, broker esecutore, clearing broker e mercato di esecuzione e se puoi una modello sql

ChatGPT

Ti suggerisco di separare nettamente i quattro ruoli, perché rappresentano aspetti diversi dell'operazione e non necessariamente soggetti diversi.

Nel tuo applicativo, che deve gestire non soltanto FX futures ma anche bond futures, interest rate futures e power futures, questa distinzione è particolarmente importante.

La prima cosa da chiarire è che **controparte, broker esecutore e clearing broker non sono necessariamente tre soggetti distinti**. Lo stesso intermediario può ricoprire più ruoli.

1. I quattro concetti

A. Controparte (Counterparty)

È il soggetto con cui il cliente conclude l'operazione dal punto di vista contrattuale.

Nel caso di un future negoziato su un mercato regolamentato, il clearing introduce la CCP (Central Counterparty), che si interpone tra le parti attraverso il processo di novazione.

Esempio:

- Cliente acquista 10 future EUR/USD.
- L'ordine viene eseguito sul CME.
- La posizione viene compensata attraverso CME Clearing.

La controparte centrale è CME Clearing, non la banca che ha trasmesso l'ordine.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Attenzione: se il cliente opera attraverso un conto omnibus, la rappresentazione giuridica e contabile dei rapporti può essere diversa. La CCP può avere come partecipante diretto il clearing member, mentre il cliente ha il rapporto contrattuale con il proprio intermediario.

B. Broker esecutore (Executing Broker)

È l'intermediario che riceve, trasmette o esegue materialmente l'ordine sul mercato.

Esempio:

- Il cliente impartisce un ordine alla banca.
- La banca lo trasmette a un broker esterno.
- Il broker esterno lo esegue sul CME.

In questo caso:

- Banca = introducing broker / intermediario del cliente.
- Broker esterno = executing broker.

Se invece la banca è direttamente abilitata all'esecuzione, può svolgere entrambi i ruoli.

C. Clearing Broker

È l'intermediario che gestisce il clearing delle operazioni attraverso la clearing house.

Per esempio, un Futures Commission Merchant (FCM) membro di CME Clearing:

- Gestisce il rapporto di clearing.
- Gestisce i margini iniziali.
- Regola il variation margin.
- Gestisce i flussi di settlement.
- Gestisce le posizioni presso la CCP.

Non necessariamente coincide con l'executing broker.

È possibile, ad esempio, che il broker A esegua l'ordine e il broker B ne gestisca il clearing.



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



D. Mercato di esecuzione (Execution Venue)

Non è una controparte e non è un intermediario: è il mercato o la piattaforma sulla quale viene eseguito l'ordine.

Esempi:

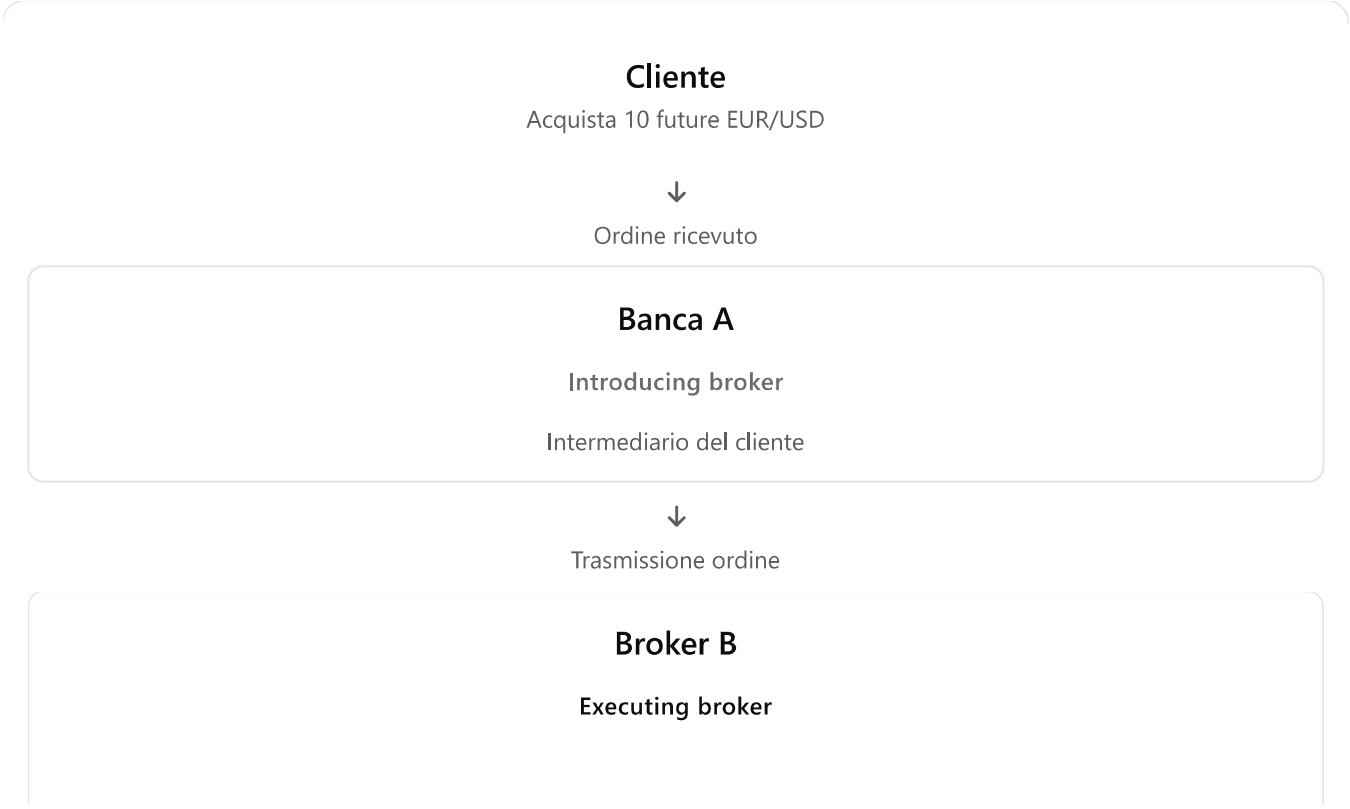
- CME Globex
- ICE Futures Europe
- Eurex
- EEX

Nel tuo caso, per il future EUR/USD, il mercato è CME.

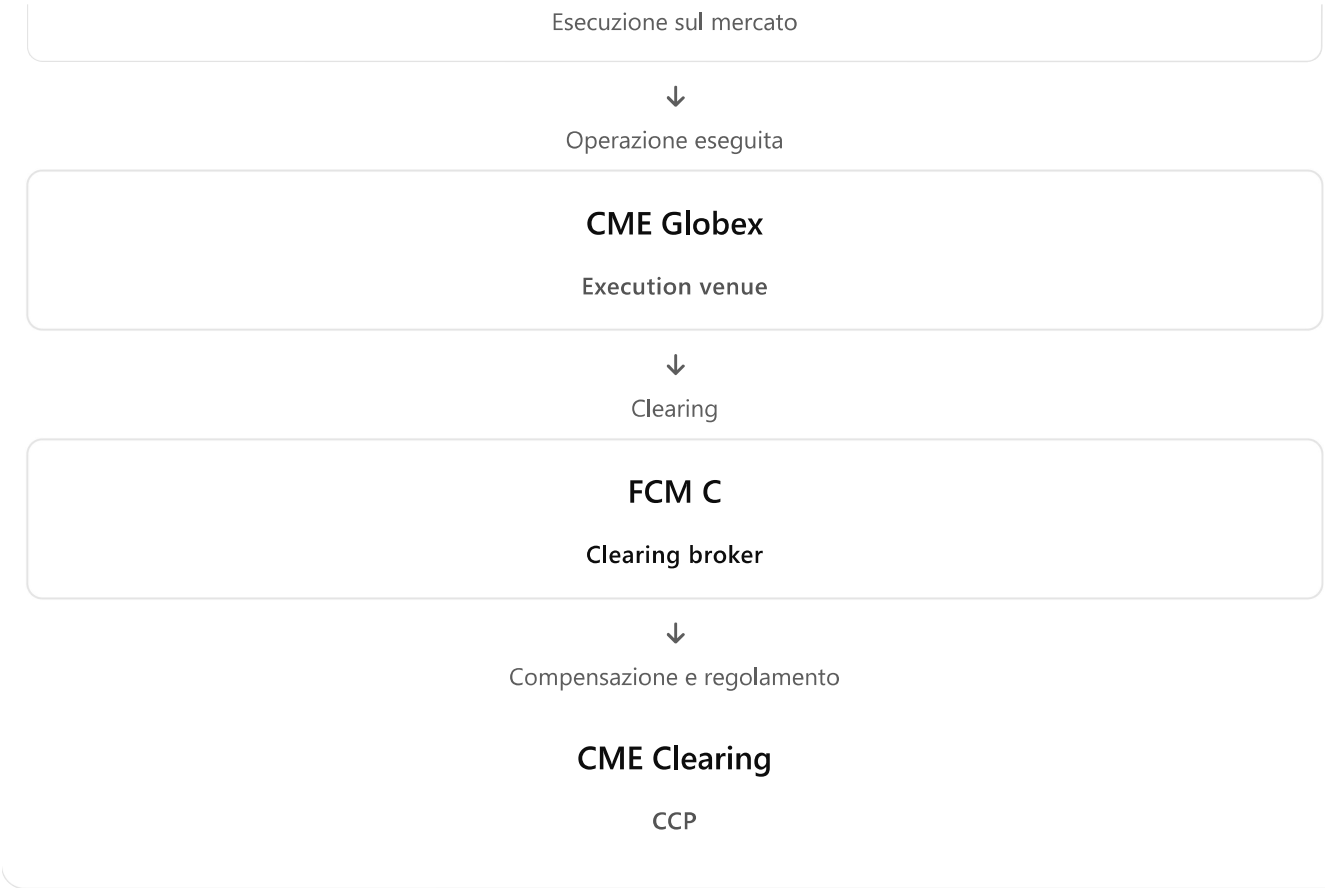
È importante distinguere il mercato di esecuzione dalla clearing house: CME Globex è la piattaforma di negoziazione, mentre CME Clearing è la CCP.

2. Un esempio completo

Immaginiamo questa struttura:



Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT



Questa è una possibile catena operativa, non l'unica.

3. Come lo modellerei nel database

Per il tuo sistema PostgreSQL/JPA, eviterei di aggiungere quattro colonne direttamente a `financial_txn` .

Il motivo è che una transazione può avere:

- Più esecuzioni a prezzi differenti.
- Più broker coinvolti.
- Un broker esecutore diverso dal clearing broker.
- Un clearing broker diverso per mercato o asset class.
- Una diversa struttura di clearing per lo stesso intermediario.

Ti propongo quindi una struttura composta da:

1. `financial_txn` : operazione finanziaria del cliente.
2. `financial_txn_execution` : singole esecuzioni dell'ordine.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

3. `financial_txn_execution_party` : soggetti coinvolti nelle esecuzioni.
4. `business_party` : anagrafica delle controparti e degli intermediari.
5. `execution_venue` : mercati e piattaforme di negoziazione.

La separazione tra transazione ed esecuzione è importante: la prima rappresenta l'operazione economica, la seconda ciò che è effettivamente accaduto sul mercato.

3.1 Anagrafica soggetti: business_party

Utilizzerei un'unica anagrafica per banche, broker, FCM, CCP e altri soggetti finanziari.

```
CREATE TABLE business_party (
    party_id        BIGINT NOT NULL,
    party_code      VARCHAR(30) NOT NULL,
    party_name      VARCHAR(150) NOT NULL,
    party_type      VARCHAR(30) NOT NULL,
    lei             VARCHAR(20),
    bic             VARCHAR(11),
    country_code    CHAR(2),
    active          BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT pk_business_party
        PRIMARY KEY (party_id),

    CONSTRAINT uk_business_party_code
        UNIQUE (party_code),

    CONSTRAINT uk_business_party_lei
        UNIQUE (lei),

    CONSTRAINT ck_business_party_type
        CHECK (party_type IN (
            'BANK',
            'BROKER',
            'FCM',
            'CCP',
            'INVESTMENT_FIRM',
            'OTHER'
```


Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT


```
        ))
);
```

Non creerei tabelle separate per `Broker`, `ClearingBroker` e `Counterparty`: sono ruoli che un soggetto può ricoprire in contesti diversi.

Per esempio, una banca può essere:

- Controparte di un derivato OTC.
- Broker esecutore di un future.
- Clearing broker di un'altra operazione.

3.2 Mercati di esecuzione: execution_venue

Terrei i mercati separati dalle anagrafiche dei soggetti.

```
CREATE TABLE execution_venue (
    venue_id        BIGINT NOT NULL,
    venue_code      VARCHAR(30) NOT NULL,
    venue_name      VARCHAR(150) NOT NULL,
    venue_type      VARCHAR(20) NOT NULL,
    mic_code        VARCHAR(4),
    country_code    CHAR(2),
    active          BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT pk_execution_venue
        PRIMARY KEY (venue_id),

    CONSTRAINT uk_execution_venue_code
        UNIQUE (venue_code),

    CONSTRAINT uk_execution_venue_mic
        UNIQUE (mic_code),

    CONSTRAINT ck_execution_venue_type
        CHECK (venue_type IN (
            'EXCHANGE',
            'MTF',
            'OTF',
```

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

```
            'ATS',
            'OTHER'
        ))
);
```

Esempi di censimento:

venue_code	venue_name	MIC
CME	CME Globex	XCME
ICE_EU	ICE Futures Europe	IFEU
EUREX	Eurex Exchange	XEUR
EEX	European Energy Exchange	XEEE

Il MIC identifica il mercato o la specifica piattaforma secondo la codifica ISO 10383. Verificherei sempre il MIC corretto per il segmento di negoziazione utilizzato.

3.3 Esecuzioni: financial_txn_execution

Questa è la tabella più importante della nuova struttura.

```
CREATE TABLE financial_txn_execution (
    execution_id        BIGINT NOT NULL,
    txn_id              BIGINT NOT NULL,

    execution_ref       VARCHAR(80),
    execution_date      TIMESTAMP NOT NULL,

    venue_id            BIGINT NOT NULL,

    execution_side      VARCHAR(4) NOT NULL,
    quantity            NUMERIC(20,8) NOT NULL,
    price               NUMERIC(20,10) NOT NULL,

    execution_status    VARCHAR(20) NOT NULL,

    created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```


Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT


```
    CONSTRAINT pk_financial_txn_execution
        PRIMARY KEY (execution_id),

    CONSTRAINT fk_execution_txn
        FOREIGN KEY (txn_id)
        REFERENCES financial_txn(txn_id),

    CONSTRAINT fk_execution_venue
        FOREIGN KEY (venue_id)
        REFERENCES execution_venue(venue_id),

    CONSTRAINT ck_execution_side
        CHECK (execution_side IN ('BUY', 'SELL')),

    CONSTRAINT ck_execution_quantity
        CHECK (quantity > 0),

    CONSTRAINT uk_execution_ref
        UNIQUE (venue_id, execution_ref)
);
```

Ho lasciato fuori alcuni campi accessori, come commissioni, valuta, execution capacity e identificativi specifici del clearing, che potrai aggiungere successivamente.

La cosa importante è che una `FinancialTxn` possa avere N esecuzioni.

Esempio:

execution_id	txn_id	quantity	price
5001	1001	4	1,08500
5002	1001	3	1,08505
5003	1001	3	1,08510

La transazione cliente rimane una sola, con quantità complessiva 10.



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



3.4 Ruoli dei soggetti: financial_txn_execution_party

Qui registrerei chi ha svolto quale funzione nell'esecuzione.

```
CREATE TABLE financial_txn_execution_party (
    execution_party_id BIGINT NOT NULL,
    execution_id       BIGINT NOT NULL,
    party_id           BIGINT NOT NULL,

    party_role         VARCHAR(30) NOT NULL,

    CONSTRAINT pk_execution_party
        PRIMARY KEY (execution_party_id),

    CONSTRAINT fk_execution_party_execution
        FOREIGN KEY (execution_id)
        REFERENCES financial_txn_execution(execution_id),

    CONSTRAINT fk_execution_party_party
        FOREIGN KEY (party_id)
        REFERENCES business_party(party_id),

    CONSTRAINT ck_execution_party_role
        CHECK (party_role IN (
            'COUNTERPARTY',
            'EXECUTING_BROKER',
            'CLEARING_BROKER',
            'INTRODUCING_BROKER',
            'CLEARING_HOUSE'
        )),

    CONSTRAINT uk_execution_party_role
        UNIQUE (execution_id, party_role)
);
```

Questa soluzione ti permette di non avere una colonna per ogni ruolo nella tabella delle esecuzioni.

Un esempio:



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



execution_id	party_id	party_role
5001	10	INTRODUCING_BROKER
5001	20	EXECUTING_BROKER
5001	30	CLEARING_BROKER
5001	40	CLEARING_HOUSE

La controparte, quando identificabile come CCP, può essere CME Clearing.

Una precisazione: la tabella proposta prevede al massimo un soggetto per ruolo e per esecuzione. Se in futuro dovessi gestire strutture con più intermediari nella stessa funzione, basterebbe eliminare il vincolo `uk_execution_party_role` e introdurre eventualmente un attributo di sequenza o livello.

4. Esempio di caricamento dati

Immaginiamo di avere questi soggetti:

```
INSERT INTO business_party
    (party_id, party_code, party_name, party_type)
VALUES
    (10, 'BANK_A', 'Banca A', 'BANK'),
    (20, 'BROKER_B', 'Broker B', 'BROKER'),
    (30, 'FCM_C', 'FCM C', 'FCM'),
    (40, 'CME_CLEARING', 'CME Clearing', 'CCP');
```

Mercato:

```
INSERT INTO execution_venue
    (venue_id, venue_code, venue_name, venue_type, mic_code)
VALUES
    (1, 'CME', 'CME Globex', 'EXCHANGE', 'XCME');
```

Supponiamo che l'esecuzione 5001 sia avvenuta sul CME.



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



```
INSERT INTO financial_txn_execution_party
    (execution_party_id, execution_id, party_id, party_role)
VALUES
    (1, 5001, 10, 'INTRODUCING_BROKER'),
    (2, 5001, 20, 'EXECUTING_BROKER'),
    (3, 5001, 30, 'CLEARING_BROKER'),
    (4, 5001, 40, 'CLEARING_HOUSE');
```

5. Come recuperare le informazioni

Una query per ottenere tutti i soggetti coinvolti in una transazione:

```
SELECT
    e.execution_id,
    e.txn_id,
    v.venue_code,
    bp.party_code,
    bp.party_name,
    ep.party_role
FROM financial_txn_execution e
JOIN execution_venue v
    ON v.venue_id = e.venue_id
JOIN financial_txn_execution_party ep
    ON ep.execution_id = e.execution_id
JOIN business_party bp
    ON bp.party_id = ep.party_id
WHERE e.txn_id = 1001
ORDER BY e.execution_id, ep.party_role;
```

Otterrai tutte le informazioni relative alla catena operativa.

6. Un'ultima scelta architetturale: dove mettere la controparte?

Qui farei una distinzione rispetto allo schema precedente.

Non tutte le controparti devono necessariamente essere collegate all'esecuzione di mercato.

Per esempio:



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



- Future CME: CCP e clearing chain sono informazioni dell'esecuzione.
- Derivato OTC: la controparte contrattuale è un'informazione della transazione.
- Bond acquistato OTC: la controparte è il venditore.
- FX spot: la controparte è il soggetto con cui viene conclusa l'operazione.

Per questo motivo, valuterei di aggiungere anche:

```
ALTER TABLE financial_txn
ADD COLUMN counterparty_id BIGINT;

ALTER TABLE financial_txn
ADD CONSTRAINT fk_financial_txn_counterparty
FOREIGN KEY (counterparty_id)
REFERENCES business_party(party_id);
```

In questo modo:

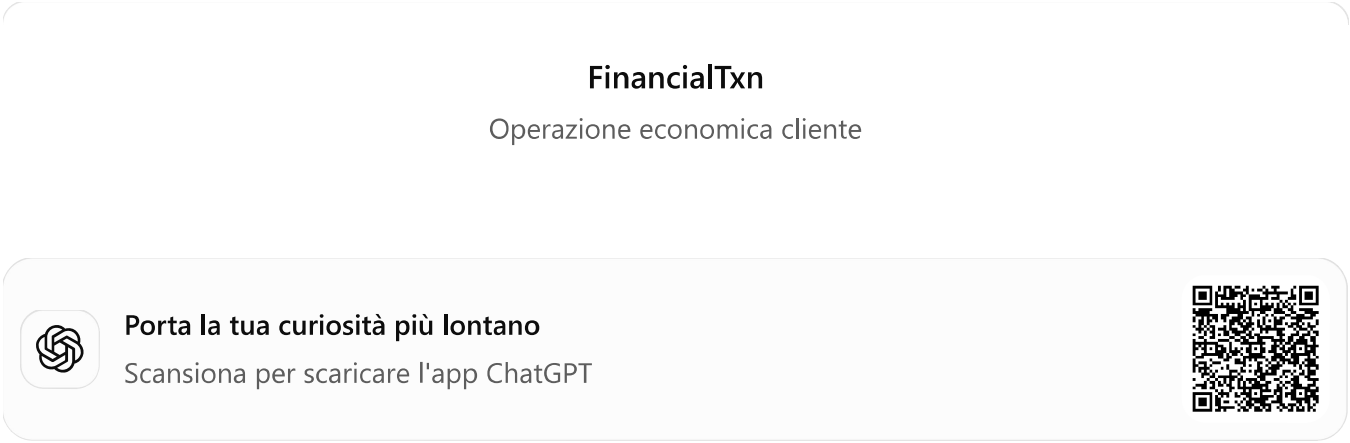
- `financial_txn.counterparty_id` identifica la controparte contrattuale della transazione.
- `financial_txn_execution_party` identifica gli intermediari e gli altri soggetti coinvolti nell'esecuzione e nel clearing.
- `execution_venue` identifica il mercato.

Nel caso del future, se il rapporto con il cliente è di intermediazione, non imposterei automaticamente la banca come controparte. La controparte e il rapporto di clearing vanno rappresentati secondo la struttura contrattuale effettiva.

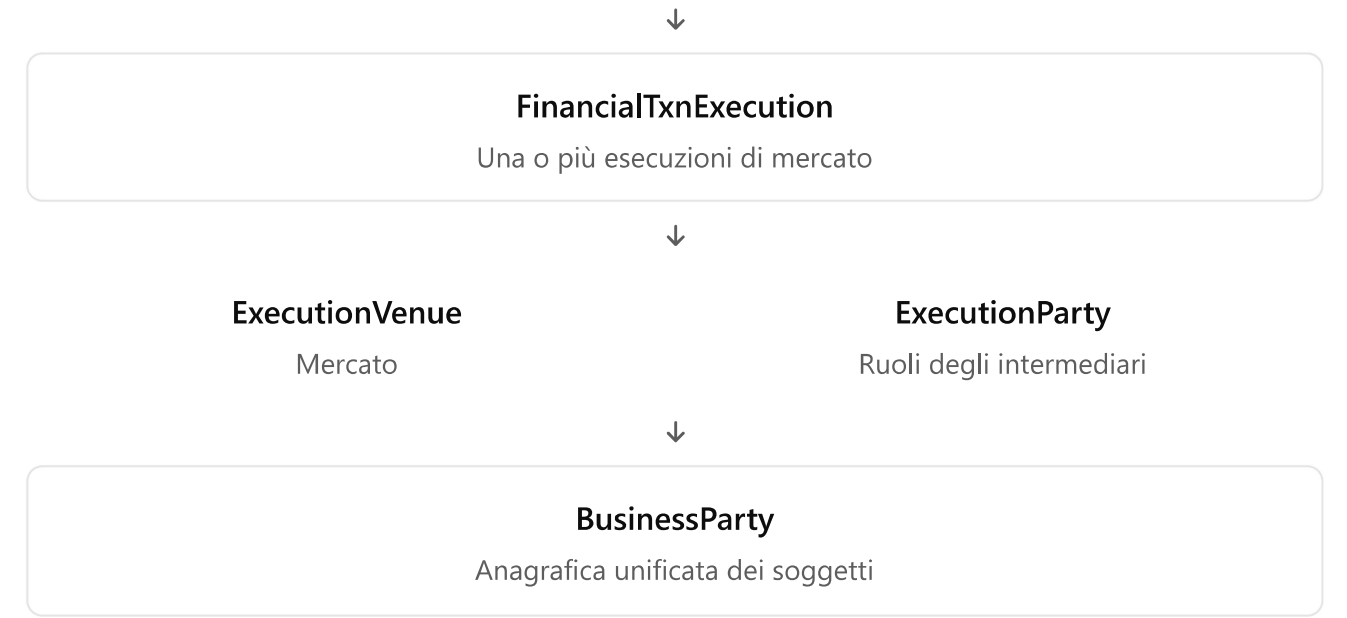
7. La mia raccomandazione per il tuo progetto

Architettura consigliata

Adotterei questa struttura:



Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT



Un vantaggio ulteriore: questa struttura è indipendente dall'asset class. Potrai riutilizzarla per futures finanziari, commodity futures, obbligazioni e operazioni FX.

Una scelta che farei subito: manterrei `FinancialTxn` come oggetto economico e introdurrei `FinancialTxnExecution` solo quando hai un'esecuzione effettiva. Non creerei una nuova transazione finanziaria per ogni fill di mercato.

È una separazione che ti tornerà molto utile anche per riconciliare le esecuzioni ricevute dal broker con gli ordini inseriti dai clienti.

Tu

Allora logicamente posso vedere le operazioni come due step separati, cioe

1. Registro l'operazione cliente (buy 10 contratti), operazione di front-office
2. Refistro esecuzione `FinancialTxnExecution` operazione di back-office

Corretto?

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

ChatGPT

Esattamente. È una separazione corretta e, soprattutto, molto vicina a come strutturerei un sistema finanziario bancario.

Distinguerei i due processi come **Front Office (Trade Capture)** e **Back Office (Trade Execution / Confirmation)**.

C'è però una precisazione: l'esecuzione può essere generata automaticamente e quasi contestualmente all'inserimento dell'ordine, oppure arrivare successivamente dal broker. I due step sono logicamente separati anche quando avvengono a pochi millisecondi di distanza.

1. Front Office – Registrazione dell'operazione cliente

Il cliente richiede l'acquisto di 10 contratti EUR/USD.

Registri una `FinancialTxn`:

Campo	Valore
txn_id	1001
portfolio	Cliente A
instrument	CME 6E DEC-26
side	BUY
quantity	10
price	NULL
status	PENDING_EXECUTION

In questa fase hai un ordine o una richiesta di negoziazione, non necessariamente un'operazione eseguita.

La banca trasmette l'ordine al broker.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

2. Back Office – Registrazione delle esecuzioni

Il broker comunica che l'ordine è stato eseguito in tre tranche:

Execution ID	Quantity	Price
5001	4	1,08500
5002	3	1,08505
5003	3	1,08510
Totale	**10**	

Registri tre `FinancialTxnExecution`, tutte collegate alla `FinancialTxn` 1001.

Il prezzo medio ponderato è:

$$P_m = \frac{4(1,08500) + 3(1,08505) + 3(1,08510)}{10}$$

$$P_m = 1,085045$$

A questo punto puoi aggiornare la `FinancialTxn`:

- `executedQuantity = 10`
- `averageExecutionPrice = 1.085045`
- `status = EXECUTED`

3. Attenzione alla gestione degli stati

Ti suggerirei di distinguere gli stati dell'ordine dagli stati delle esecuzioni.

FinancialTxn

```
public enum TxnStatus {
    PENDING_EXECUTION,
    PARTIALLY_EXECUTED,
    EXECUTED,
```

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

```
    CANCELLED,
    REJECTED
}
```

FinancialTxnExecution

```
public enum ExecutionStatus {
    CONFIRMED,
    CANCELLED,
    CORRECTED
}
```

Le esecuzioni devono essere gestite autonomamente: per esempio, un'esecuzione può essere successivamente cancellata o corretta dal broker.

Inoltre, `EXECUTED` dovrebbe significare che tutta la quantità dell'ordine è stata eseguita, non semplicemente che è arrivato il primo fill.

4. Il vantaggio per la tua architettura

Questa separazione ti permette di avere tre momenti distinti:

1. **Front Office – Trade Capture**

   Registrazione della richiesta del cliente e trasmissione dell'ordine.

2. **Back Office – Execution Processing**

   Ricezione, registrazione e riconciliazione dei fill del broker.

3. **Position Engine – Position Update**

   Aggiornamento della posizione cliente sulla base delle quantità effettivamente eseguite.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Quest'ultimo punto è importante: nel modello broker, non aggiornerei la posizione definitiva per tutti i 10 contratti al momento dell'inserimento dell'ordine. La aggiornerei progressivamente in base alle esecuzioni confermate.

Un ulteriore suggerimento: manterrei anche la quantità originariamente ordinata (`orderedQuantity`) separata dalla quantità eseguita (`executedQuantity`). Ti servirà per gestire esecuzioni parziali, cancellazioni dell'ordine residuo e modifiche della quantità.

Questa impostazione si integra molto bene con il tuo attuale `FinTxnPollingJob` e con il meccanismo di generazione degli `AccountingEvent` .

Tu

A questo punto mi sembra che l'ordine iniziale del cliente sia un oggetto a se stante(anche come tabella). Da questo ordine, poi confermato, nacono le financialTxn, ogniuna con la sua quantity e prezzo

ChatGPT

Sì. Anzi, questa evoluzione mi convince più della struttura che avevamo ipotizzato inizialmente con `FinancialTxn` come ordine e `FinancialTxnExecution` come singola esecuzione.

Stai arrivando a una distinzione architetturale molto importante:

Ordine, esecuzione e transazione finanziaria sono tre oggetti distinti, ciascuno con una propria responsabilità.

Nel tuo applicativo li modellerei così:

1. `FinancialOrder` : rappresenta l'intenzione del cliente di acquistare o vendere uno strumento.
2. `FinancialTxn` : rappresenta l'operazione finanziaria effettivamente conclusa.
3. `FinancialTxnExecution` : rappresenta il dettaglio tecnico dell'esecuzione sul mercato.

1. Il flusso operativo

Immaginiamo che il cliente richieda l'acquisto di 10 future EUR/USD.



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



1. FinancialOrder

Front Office

- BUY 10
- CME 6E DEC-26
- Prezzo limite: 1,08500 (eventuale)
- Stato: SUBMITTED

↓

Trasmissione al broker

2. FinancialTxn

Trade Capture

Nascono tre transazioni dalle esecuzioni ricevute:

- BUY 4 @ 1,08500
- BUY 3 @ 1,08505
- BUY 3 @ 1,08510

↓

Dettaglio esecutivo

3. FinancialTxnExecution

Back Office

Conserva i dettagli delle esecuzioni comunicate dal broker, gli identificativi di mercato, gli intermediari e i dati di clearing.

In questo schema, ogni fill genera una `FinancialTxn` distinta, con quantità e prezzo propri.

Questo è particolarmente utile perché ogni transazione può avere:

- Una propria data di negoziazione.
- Un proprio prezzo.
- Una propria quantità.
- Una propria controparte o struttura di clearing, se necessario.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT



- Un proprio ciclo contabile.

2. Relazione tra le tabelle

La struttura sarebbe:

```
FinancialOrder
      |
      | 1:N
      |
FinancialTxn
      |
      | 1:1 oppure 1:N
      |
FinancialTxnExecution
```

Qui però farei una considerazione importante.

Se ogni `FinancialTxn` nasce da una singola esecuzione (fill), **non è strettamente necessario mantenere anche** `FinancialTxnExecution`.

Infatti, avresti:

- Una `FinancialOrder` con quantità ordinata pari a 10.
- Tre `FinancialTxn`, ciascuna corrispondente a un fill.
- I dati dell'esecuzione (execution ID, venue, broker, prezzo, quantità) direttamente nella `FinancialTxn`.

La tabella `FinancialTxnExecution` diventa utile se vuoi mantenere separati il messaggio originale di esecuzione ricevuto dal broker e la transazione finanziaria normalizzata e contabilizzata.

Io partirei senza questa ulteriore tabella, salvo esigenze di riconciliazione particolarmente articolate.

3. Proposta di modello SQL

financial_order

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

```
CREATE TABLE financial_order (
    order_id            BIGINT PRIMARY KEY,
    portfolio_id        BIGINT NOT NULL,
    instrument_id       BIGINT NOT NULL,

    order_side          VARCHAR(4) NOT NULL,
    order_type          VARCHAR(20) NOT NULL,

    ordered_quantity    NUMERIC(20,8) NOT NULL,
    limit_price         NUMERIC(20,10),

    executed_quantity   NUMERIC(20,8) NOT NULL DEFAULT 0,

    order_status        VARCHAR(30) NOT NULL,

    created_at          TIMESTAMP NOT NULL,
    submitted_at        TIMESTAMP,
    completed_at        TIMESTAMP
);
```

financial_txn

Manterresti la tua tabella attuale, aggiungendo il riferimento all'ordine:

```
ALTER TABLE financial_txn
ADD COLUMN order_id BIGINT;

ALTER TABLE financial_txn
ADD CONSTRAINT fk_financial_txn_order
FOREIGN KEY (order_id)
REFERENCES financial_order(order_id);
```

Ogni transazione rappresenta un'esecuzione effettiva.

Aggiungerei anche:



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



```
ALTER TABLE financial_txn
ADD COLUMN execution_ref VARCHAR(80);

ALTER TABLE financial_txn
ADD COLUMN venue_id BIGINT;

ALTER TABLE financial_txn
ADD COLUMN executing_broker_id BIGINT;
```

Gli altri ruoli operativi possono essere gestiti con la struttura dei soggetti che abbiamo definito in precedenza.

4. Ciclo di vita dell'ordine

Prevederei questi stati:

```
public enum OrderStatus {
    NEW,
    SUBMITTED,
    PARTIALLY_FILLED,
    FILLED,
    PARTIALLY_CANCELLED,
    CANCELLED,
    REJECTED,
    EXPIRED
}
```

Esempio:

Evento	Quantità eseguita	Stato
Ordine inserito	0	NEW
Ordine trasmesso	0	SUBMITTED
Primo fill	4	PARTIALLY_FILLED
Secondo fill	7	PARTIALLY_FILLED

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

Terzo fill	10	FILLED

Se il cliente annulla l'ordine dopo il primo fill, la situazione sarà:

- Quantità ordinata: 10
- Quantità eseguita: 4
- Quantità cancellata: 6
- Stato: PARTIALLY_CANCELLED

La transazione relativa ai 4 contratti rimane valida.

5. Un vantaggio importante per il tuo motore contabile

Questa architettura si integra molto bene con il sistema che stai sviluppando.

La sequenza diventa:

1. `FinancialOrder` : nessun movimento di posizione e nessun evento contabile di trade.
2. Arrivo del fill: creazione di `FinancialTxn` .
3. `FinancialTxn` passa a EXECUTED.
4. Generazione di `AccountingEvent.TRADE_EXECUTED` .
5. Aggiornamento della `PositionDetail` .
6. Gestione successiva di margini, variation margin e settlement.

In questo modo il motore contabile non deve conoscere gli ordini: lavora esclusivamente sulle operazioni effettivamente eseguite.

La mia scelta sarebbe quindi: `FinancialOrder 1:N FinancialTxn` **, senza** `FinancialTxnExecution` **nella prima implementazione.**

È un modello pulito, evita duplicazioni e si integra con la tua attuale architettura di `FinancialTxn` , `PositionDetail` e `AccountingEvent` .

Tu

A questo punto vedrei una struttura di questo tipo

1. FinalcialOrder es 10 Fut EUR/USD buy, proposedPrice 1.125 (prezzo limite), counterparty Client

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

2. FinancialTxn 4 + 3 + 3 Fut EUR/USD buy, prz1 prz2 prz3, counterparty Client, riferimento FinalcialOrder
3. FinancialExecution 1-1 con FinancialTxn controparte Client , executor CME

ChatGPT

Sì, la struttura è coerente e, soprattutto, introduce una distinzione importante tra **ordine**, **trade concluso** ed **esecuzione sul mercato**.

La modellerei proprio con queste tre entità, ma farei una precisazione sul significato di `FinancialExecution` : non la considererei una seconda transazione finanziaria, bensì il dettaglio dell'esecuzione che ha prodotto la `FinancialTxn` .

Nel tuo esempio:

- `FinancialOrder` : il cliente ordina di acquistare 10 futures EUR/USD, con limite 1,125.
- `FinancialTxn` : vengono conclusi tre trade, rispettivamente 4, 3 e 3 contratti, ciascuno al proprio prezzo.
- `FinancialExecution` : registra per ciascun trade i dettagli dell'esecuzione, ad esempio CME come mercato e il broker/intermediario che l'ha eseguita.

La cardinalità che proponi è quindi:

```
FinancialOrder 1:N FinancialTxn 1:1 FinancialExecution
```

1. Attenzione alla distinzione tra Client, Counterparty e Executor

Qui farei una correzione terminologica importante.

Il **Client** è il soggetto per conto del quale viene inserito l'ordine. Non lo definirei `Counterparty` , perché non è necessariamente la controparte del trade.

Per un future negoziato su CME distinguerei:

Ruolo	Esempio
Client	Cliente titolare dell'ordine



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



Executing Broker	La banca o il broker che trasmette/esegue l'ordine
Execution Venue	CME Globex
Clearing Broker / FCM	Intermediario membro della clearing house
Clearing House / CCP	CME Clearing

Quindi, nel tuo esempio, **CME Globex è la Execution Venue**, non l'Executor. CME Clearing è invece la controparte centrale nel processo di clearing.

Terrei questi ruoli distinti nel modello, anche se in alcuni casi la stessa entità può ricoprirne più di uno.

2. Le tre entità

La struttura potrebbe essere questa:

FinancialOrder

Istruzione del cliente

- Client: Cliente A
- Instrument: EUR/USD Future
- Side: BUY
- Quantity: 10
- OrderType: LIMIT
- LimitPrice: 1.125

↓

1 : N

FinancialTxn

Trade effettivamente conclusi

- Txn 1: BUY 4 @ 1.1248
- Txn 2: BUY 3 @ 1.1250
- Txn 3: BUY 3 @ 1.1249

↓

Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT

1 : 1

FinancialExecution

Dettagli dell'esecuzione

- ExecutionRef
- ExecutionTimestamp
- ExecutionVenue: CME Globex
- ExecutingBroker
- ClearingBroker / FCM
- Eventuale ClearingHouse

La cardinalità `FinancialTxn 1:1 FinancialExecution` è corretta se ogni tua transazione rappresenta esattamente un fill.

3. Come organizzerei le responsabilità

- `FinancialOrder` : gestisce il ciclo di vita dell'ordine (`NEW` , `SUBMITTED` , `PARTIALLY_FILLED` , `FILLED` , `CANCELLED` , `REJECTED`).
- `FinancialTxn` : rappresenta il singolo trade e alimenta il tuo motore delle posizioni e quello contabile.
- `FinancialExecution` : conserva le informazioni operative e di mercato relative al fill.

Un dettaglio importante: se in futuro un singolo fill di mercato dovesse essere ripartito tra più clienti o portafogli, la cardinalità diventerebbe `FinancialExecution 1:N FinancialTxn` . Per questo motivo manterrei la tabella separata, anche se inizialmente la relazione sarà 1:1.

La mia raccomandazione: manterrei questa architettura. È sufficientemente semplice per partire, ma non ti vincola a un modello esclusivamente 1:1 tra esecuzione di mercato e trade contabilizzato.

Inoltre, distinguerei nel database `client_id` da `counterparty_id` : il primo identifica il cliente cui appartiene l'ordine e la posizione; il secondo identifica la controparte contrattuale effettiva, quando applicabile.

Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT

Tu

Si ma il cliente e'una controparte, quello che forse manca su controparte e`la tipologia, es cliente, brooker,...

ChatGPT

Sì, hai ragione. Nel tuo modello il cliente può tranquillamente essere una **controparte**. La mia distinzione precedente era troppo rigida: il problema non è il concetto di controparte, ma il fatto che manca una classificazione del suo ruolo.

Io introdurrei un'entità unica `BusinessParty` (o manterrei il tuo `Counterparty`, se è già presente), con una tipologia.

Per esempio:

Controparte	Tipologia
Cliente A	CLIENT
Banca XYZ	BANK
Broker ABC	BROKER
FCM DEF	CLEARING_BROKER
CME Clearing	CCP
EEX	EXCHANGE

Attenzione: distinguerei la **tipologia dell'entità** dal **ruolo che assume nella singola operazione**. Un broker può essere anche executing broker e clearing broker, a seconda dell'operazione.

Come lo applicherei alle tue entità

- `FinancialOrder.counterparty_id` : cliente che ha impartito l'ordine.
- `FinancialTxn.counterparty_id` : cliente con cui è registrato il trade.
- `FinancialExecution.execution_venue_id` : mercato sul quale è avvenuta l'esecuzione.
- `FinancialExecution.executing_broker_id` : broker che ha eseguito l'ordine.
- `FinancialExecution.clearing_broker_id` : intermediario che gestisce il clearing.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT

In questo modo puoi riutilizzare la stessa anagrafica delle controparti in tutte le entità.

Un'ulteriore scelta progettuale: nella tabella `Counterparty` userei un campo `party_type` con valori come `CLIENT`, `BANK`, `BROKER`, `CLEARING_BROKER`, `CCP`, `EXCHANGE`, `OTHER`.

Non userei invece `party_type` per descrivere il ruolo nella singola esecuzione: quello lo manterrei nei riferimenti di `FinancialExecution`.

Così il modello resta normalizzato e puoi gestire anche il caso in cui la tua banca sia contemporaneamente executing broker e clearing broker.

Tu

quindi la chiave logica della tabella counterparty dovrebbe essere counterparty_id e party_type?

ChatGPT

No, manterrei `counterparty_id` **come unica chiave primaria**. Non includerei `party_type` nella chiave.

La distinzione è importante:

- `counterparty_id`: identifica univocamente il soggetto giuridico.
- `party_type`: classifica il soggetto, ma non ne determina l'identità.

Ad esempio:

counterparty_id	name	party_type
1	Cliente A	CLIENT
2	Broker ABC	BROKER
3	CME Clearing	CCP
4	Banca XYZ	BANK

La struttura sarebbe:



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



```
CREATE TABLE counterparty (
    counterparty_id BIGINT PRIMARY KEY,
    party_code VARCHAR(30) NOT NULL UNIQUE,
    party_name VARCHAR(150) NOT NULL,
    party_type VARCHAR(30) NOT NULL,
    lei VARCHAR(20),
    bic VARCHAR(11),
    country_code CHAR(2),
    active BOOLEAN NOT NULL DEFAULT TRUE
);
```

La chiave logica alternativa potrebbe essere `party_code`, purché sia un codice univoco e stabile.

Un'ultima considerazione: se un soggetto può assumere più tipologie contemporaneamente (ad esempio una banca che opera anche come broker), valuterei una relazione `counterparty 1:N counterparty_role`, anziché un singolo `party_type`.

Per partire, comunque, il singolo campo `party_type` è perfettamente ragionevole.

Tu

Scusa mio errore, counterparty_id epkey, `chiave logica` eparty_code e party_type, cosi la stessa controparte puo avere piu ruoli

ChatGPT

Esatto, ora è chiaro. Vuoi che la stessa entità possa comparire più volte nell'anagrafica, una per ciascuna tipologia/ruolo.

In questo caso la struttura:

- `counterparty_id`: primary key tecnica.
- `(party_code, party_type)`: unique constraint, cioè chiave logica composta.

È assolutamente valida.



Porta la tua curiosità più lontano

Scansiona per scaricare l'app ChatGPT



Ad esempio:

counterparty_id	party_code	party_name	party_type
1	CME	CME Group	EXCHANGE
2	CME	CME Group	CLEARING_HOUSE
3	JPM	JP Morgan	BANK
4	JPM	JP Morgan	BROKER
5	CLI001	Cliente A	CLIENT

Vincolo:

```
CONSTRAINT uk_counterparty_code_type
    UNIQUE (party_code, party_type)
```

Ti suggerirei però una piccola evoluzione concettuale: se vuoi evitare di duplicare nome, LEI, BIC, indirizzo e altri dati anagrafici, sarebbe più pulito separare:

- `BusinessParty` : identifica il soggetto giuridico.
- `CounterpartyRole` : associa al soggetto uno o più ruoli.

In questo modo JPM esiste una sola volta, ma può avere i ruoli BANK e BROKER.

Per la tua applicazione, dove le controparti sono utilizzate in ordini, transazioni ed esecuzioni, preferirei questa seconda soluzione nel lungo periodo. La prima resta comunque una buona soluzione pragmatica se vuoi mantenere un modello più semplice.

Porta la tua curiosità più lontano
Scansiona per scaricare l'app ChatGPT